

Cotizador instantáneo de piezas de manufactura con algoritmos de Machine Learning.

Imanol Muñiz Ramirez A01701713

Contenido

Cotizador instantáneo de piezas de manufactura con algoritmos de Machine Learning.	1
Contexto	1
Primera iteración	2
Extracción de los datos.....	2
Transformación de los datos	4
Carga de los datos	7
Modelado	8
Segunda iteración.....	11
Extracción de los datos.....	11
Transformación de los datos	11
Carga de los datos	12
Modelado	12
Próximos pasos	14
Consideraciones	14
Bibliografía.....	15

Contexto

Radii es una startup mexicana que se dedica a manufacturar piezas sin tener una sola máquina. Opera como un punto intermedio entre los clientes que quieren alguna pieza y los talleres que cuentan con la maquinaria y procesos de fabricación. Radii se encarga de toda la gestión de cotizaciones, traslados, problemas de calidad, especificaciones etc. Para que el cliente solo se tenga que preocupar por pedir y obtener lo que necesita.

Uno de los problemas que estoy resolviendo dentro de esta organización es cómo podemos hacer para que el cliente tenga una cotización de la pieza al instante debido a que

actualmente, cada pieza es evaluada por un ingeniero para obtener su complejidad, el tipo de máquinas que requiere, cuánto tiempo tomará, que proveedor puede fabricarla, entre otras cosas y luego requiere hacer otras cotizaciones de la materia prima y del costo de fabricación con los talleres o “*partners*”. Esto hace que la respuesta de Radii al cliente pueda tomar días.

Para atender esta situación Radii se planteó desarrollar 3 herramientas tecnológicas para inferir los precios de las piezas: feasibility AI, que sirve para extraer información de aquellas piezas que tienen planos o dibujos técnicos, cotizador 2D, que con base en los datos extraídos calcula el precio y finalmente el cotizador 3D que tiene dos vertientes de desarrollo, un cálculo matemático y un modelo de *Machine Learning*, ambos con base en los datos que se extraen del modelo 3D de la pieza por medio de un “*Design for Manufacturing (DFM) analysis*” automatizado y donde es en este último enfoque lo que estoy desarrollando así como el motivo de este documento.

Primera iteración

Objetivo: Inferir el precio de la pieza que nos da el partner,

Justificación: Inferir correctamente el precio que nos dan los partners nos permite únicamente tener que añadir matemáticamente el margen y costos logísticos de Radii. Podemos obtener los datos que determinan el costo de fabricación de una pieza a través del análisis de los ingenieros y el DFM.

Carpeta: iter1

Extracción de los datos

Esta sin duda ha sido está siendo la parte más ardua del trabajo. Radii al ser una startup no tiene los procesos ni las herramientas totalmente definidas para estar constantemente almacenando la información, por lo que para conseguir una cantidad informativa de datos fue necesario realizar diversas tareas:

1. Extracción de datos históricos de OneDrive: Cada ingeniero maneja proyectos de piezas a su manera. Esto provoca que la información esté dispersa o incompleta. Durante 4 semanas se recabaron datos de las cotizaciones de los partners que almacenaban en el OneDrive de la organización y se subieron a la plataforma. Se logró rescatar la información completa de 81 casos y parcial de 98 más.
2. Creación de herramientas en la plataforma para que los ingenieros capturen la información: Se desarrollaron formularios dentro de la plataforma web de la organización para la captura de los datos de las piezas o proyectos.

3. Creación de herramientas para la exportación de los datos: Se desarrollaron herramientas dentro de la plataforma web para descargar los datos registrados con el objetivo de poder analizarlos.
4. Creación de procedimientos de extracción de datos: Se propuso un esquema de trabajo basado en procedimientos y estándares que garantice el llenado correcto y completo de los datos.

Cabe destacar que muchas de las actividades anteriores siguen en proceso o en mejora. Constantemente se añade un dato que se tiene que llenar, un cambio en el proceso, obtenemos más datos del drive u otras fuentes. Para la fecha de corte (11 de marzo del 2026) obtuvimos un total de 588 registros de piezas cotizadas por los partners con 91 columnas que describen geometrías, tamaños, tiempo de fabricación, tecnologías requeridas, identificadores etc.

Columnas: manufacturing_quote_id, manufacturing_quote_created_at , line_item_id, radii_id, source_type, product_name, actual_partner_quoted_price, actual_partner_quoted_quantity, partner_id, partner_name, partner_country, quote_is_awarded, part_volume_cm3, weight_g , stock_volume_cm3, removed_volume_cm3, material_removal_pct, stock_dimensions_mm.x, stock_dimensions_mm.y, stock_dimensions_mm.z, faces_qty, aspect_ratio, thin_ratio, plane_ratio, cyl_ratio, complex_ratio, total_holes, machining_hours, programming_hours, num_setups, setup_hours, setup_time_per_setup, roughing_hours, finishing_hours, material_dependent_hours, effective_hourly_rate, complexity_factor, mesh_complexity_penalty, complexity_risk_mult, total_risk_mult, tolerance_tier, tolerance_tier_mult, tolerance_risk_mult, axis_requirement, edm_required, part_type , material_cost, machining_cost, feature_cost, finishing_cost, heat_treatment_cost, post_production_cost, overhead, total_cost_floor, quantity, material_nesting_factor, machining_batch_factor, overhead_batch_factor, feature_machining_hours, tool_change_hours, amortized_fixed_hours, material_title, technology, tolerance, finishing, heat_treatment, is_manufacturable, errors_count, warnings_count, summary.critical, summary.medium, summary.low, summary.high, direction_count, min_setups_3axis, setup_complexity, undercut_count, angled_hole_count, inaccessible_feature_count, axis_recommendation, face_orientation_distribution.X+, face_orientation_distribution.X-, face_orientation_distribution.Y+, face_orientation_distribution.Y-, face_orientation_distribution.Z+, face_orientation_distribution.Z-, metadata.tolerance_mm, metadata.max_hole_depth_mm, metadata.bounding_box_mm.x, metadata.bounding_box_mm.y, metadata.bounding_box_mm.z

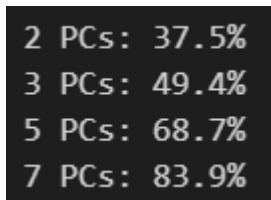
Transformación de los datos

Tenía la idea de que existieran patrones en la geometría de las piezas que dictaminara el precio, por lo que en el notebook 01_exploration.ipynb realicé un análisis para determinar qué registros y qué columnas tenían datos geométricos para inferir el precio de fabricación del partner.

Empecé por eliminar aquellos registros que no tuvieran análisis DFM, desafortunadamente un número muy alto de proyectos no cuentan con modelos 3D por lo que no es posible realizar este análisis, por lo que se eliminaron 289 registros dejando un total de 297.

Sobre las columnas directamente eliminé aquellas que no describieran la geometría de la pieza, como tiempos de fabricación, tecnologías a utilizar, identificadores, etc. Posteriormente realicé un análisis para determinar aquellas columnas geométricas que no aporten información debido su invariabilidad entre registros o que estén muy correlacionadas con alguna otra columna, por ejemplo, face_orientation_distribution.X- en la que todos los registros tienen 0 o thin_ratio y bbox_ratio_3 que ambas describen que tan plana es una pieza.

En 02_clustering.ipynb hice algunos intentos de agrupar las piezas para desarrollar modelos de predicción más específicos dependiendo del tipo de pieza. En “Radii parts clusters.pdf” muestro los resultados. Con base en el análisis del paso anterior, determiné qué columnas eran numéricas e hice una reducción de dimensionalidad con PCA para ver cuántos PCs eran necesarios para poder comprender la información en un plano 2D.



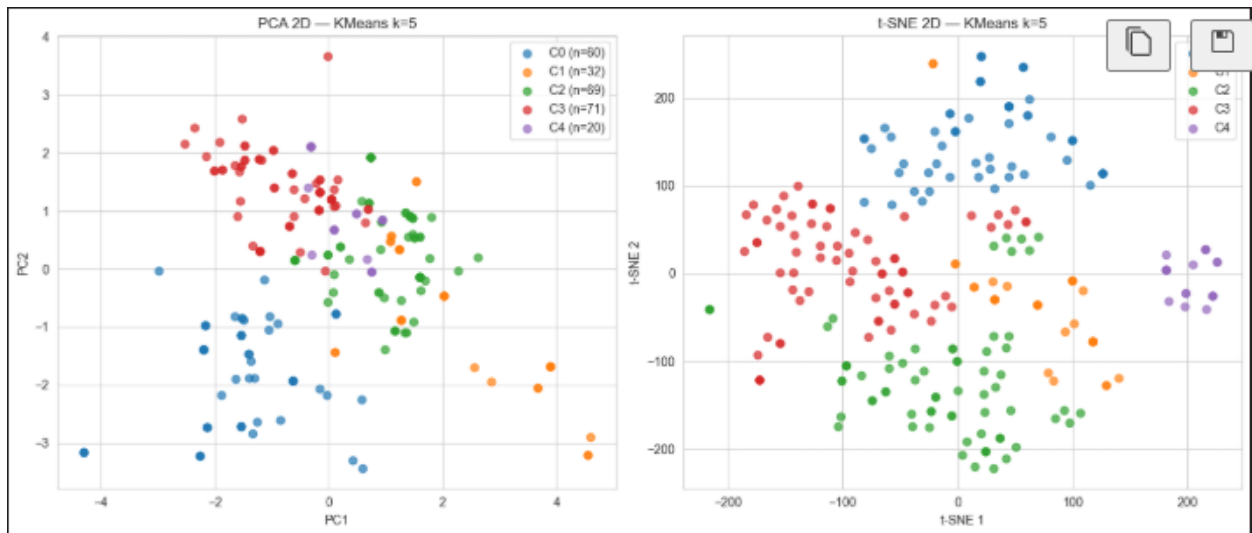
2 PCs:	37.5%
3 PCs:	49.4%
5 PCs:	68.7%
7 PCs:	83.9%

Desafortunadamente solo con 2 PCs se explicaba el 37.5% por lo que no podríamos ver en los gráficos claramente las relaciones.

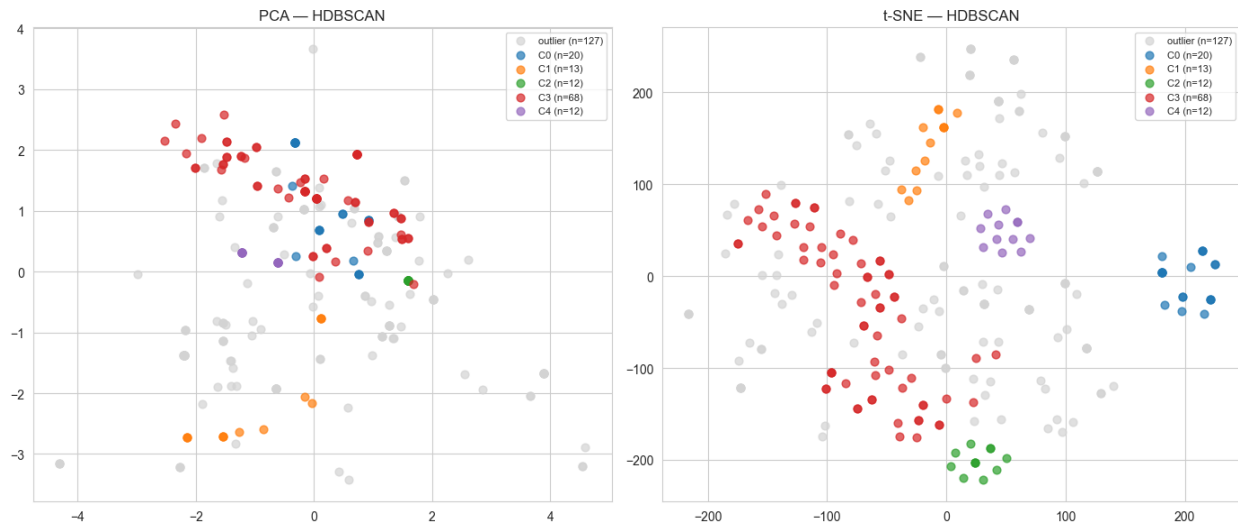
Proseguí a verificar cuál sería el K más indicado para hacer el agrupamiento por K-means a través del método de silhouette y obtuve K=10 como la mejor opción, sin embargo, pensar en categorizar los registros en K=10 dejaría muy pocos en cada categoría, pensé que el máximo sería K=5 para en promedio tener 60 registros en cada una. Otra opción viable era K=3 así que probé con ambas, aunque al final me quedé con K=5.

```
k=2: silhouette=0.204
k=3: silhouette=0.229
k=4: silhouette=0.178
k=5: silhouette=0.213
k=6: silhouette=0.249
k=7: silhouette=0.265
k=8: silhouette=0.248
k=9: silhouette=0.256
k=10: silhouette=0.301
```

Mejor k por silhouette: 10



También lo hice con HDBSCAN para detectar outliers, con un mínimo de 5 grupos con 10 ejemplos. Resultó que con esta configuración el 50.4% de los datos fueron detectados como outliers, lo que podría reflejar una variación muy alta en las características de las piezas.

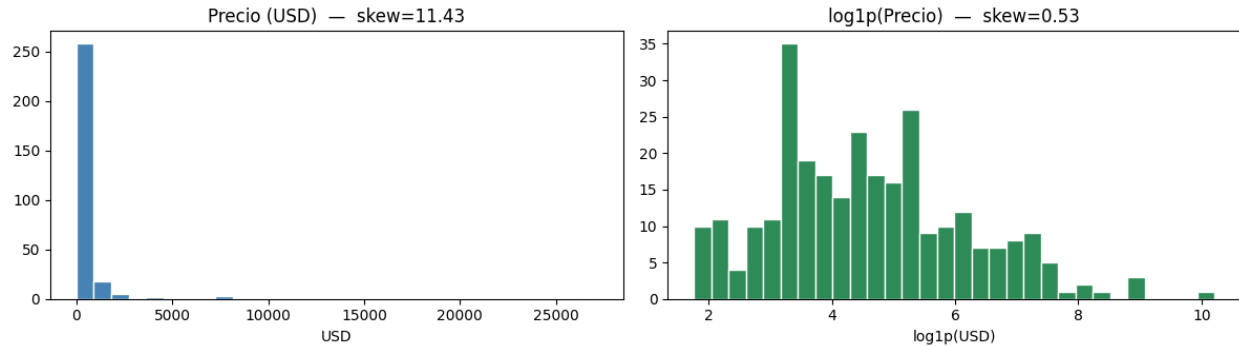


El agrupamiento de las piezas para realizar modelos específicos de cada categoría fue analizada por un experto en el área de manufactura y concluimos que entre las piezas no existía una relación geométrica que dictaminara el tiempo o la tecnología de fabricación por lo que fue una idea descartada.

A pesar de los resultados anteriores, en el notebook 03_clen_dataset.ipynb, utilizamos la información obtenida en los pasos anteriores para realizar la eliminación de las columnas que no son útiles y procedí a analizar aquellas que no habían sido contempladas pues no eran información geométrica como por ejemplo país de fabricación, acabados, post procesamientos etc. Esto con el objetivo de crear un dataset para un modelo general, que no esté enfocado a un cierto tipo de geometría, tecnología, partner o alguna otra variable como estaba considerando hacerlo antes.

Adicionalmente quitamos registros con precios o cantidades posiblemente incorrectos y outliers.

Finalmente, en 05_price_model.ipynb quitamos las columnas de identificadores, separamos la variable objetivo y la normalizamos con la función $\log_{1p}$ debido a que la distribución de precios está muy cargada a la izquierda y provocaría que las predicciones del modelo se comporten de forma similar infiriendo casi todo hacia los precios que están concentrados los datos y equivocándose enormemente en piezas raras. Al normalizar obtenemos una curva más suave en la que los errores no serán tan drásticos.



Al final de la transformación de los datos obtuvimos un dataset con 288 registros y 22 columnas.

Columnas finales:

```
CATEGORICAL_FEATURES = [
    'partner_country',
    'material_title',
    'technology',
    'tolerance_tier',
    'finishing',
    'heat_treatment',
    'axis_recommendation',
]

NUMERIC_FEATURES = [
    'actual_partner_quoted_quantity',
    # Shape features dimensionless
    'aspect_ratio', 'plane_ratio', 'cyl_ratio', 'complex_ratio',
    'bbox_ratio_2', 'bbox_ratio_3',
    'face_orientation_distribution.X+_pct',
    'face_orientation_distribution.Y+_pct',
    'direction_count',
    'angled_hole_count_log',
    # Tamaño absoluto (solo un representante)
    'part_volume_cm3',
    # Conteos geométricos
    'total_holes', 'faces_qty', 'metadata.max_hole_depth_mm',
]
```

Carga de los datos

Para cargar los datos al modelo eficientemente se pueden reutilizar los notebooks y extraer los datos de la herramienta de extracción que se hizo en la plataforma de la organización.

Sin embargo, sería necesario hacer adecuaciones en caso de que los datos extraídos tengan nuevas columnas que quieran considerarse.

Modelado

Sobre el modelo

El modelo seleccionado fue CatBoost que es un *gradient boost decision tree* debido a su gran capacidad para manejar datos tabulares sin normalizaciones como *one hot encoding* o pipelines complejos, con bajo *information leakage* y su rapidez de entrenamiento y respuesta. Ambas consideraciones son claves para el contexto del problema pues el dataset de las piezas de manufactura contiene una gran cantidad de columnas categóricas cómo el acabado de la pieza, tratamientos térmicos o país de fabricación. A su vez, dado que el mercado de piezas maquinadas cambia constantemente, debido a la varianza de los precios de materiales, relaciones internacionales, los cambios en la oferta y demanda es importante contar con un modelo que sea rápido de entrenar y dar respuestas.

Aunque no existen (o no pude encontrar) investigaciones dónde se use este modelo específicamente para este propósito, en muchas otras áreas se a probado la eficiencia del algoritmo en problemas que implican el manejo de variables categóricas y cómo destaca por su velocidad, facilidad y precisión comparado con otros esquemas del “*state of the art*”:

“Mambular: A Sequential Model for Tabular Deep Learning”

Model	BH ↓	CW ↓	FF ↓	GS ↓	HI ↓	K8 ↓	AV ↓	KC ↓	MH ↓	NP ↓	PP ↓	SA ↓	SG ↓	VT ↓	Rank ↓
Mambular	0.021	0.701	0.272	0.057	0.595	0.168	0.018	0.137	0.085	0.003	0.402	0.015	0.318	0.003	1.79
FTTransformer	0.028	0.701	0.301	0.205	0.609	0.451	0.089	0.149	0.101	0.009	0.542	0.033	0.36	0.045	4.36
CatBoost	0.032	0.702	0.245	0.041	0.597	0.150	0.004	0.110	0.078	0.005	0.39	0.018	0.297	0.013	1.79
LightGBM	0.048	0.707	0.263	0.059	0.599	0.239	0.024	0.140	0.091	0.009	0.452	0.031	0.302	0.013	3.26
XGBoost	0.039	0.752	0.281	0.078	0.635	0.259	0.004	0.161	0.098	0.006	0.403	0.024	0.329	0.013	3.71

(Boncinelli et al., 2025)

“A Comprehensive Evaluation of CatBoost and LightGBM Algorithms for Honorarium Prediction on Categorical Datasets with Class Imbalance”

TABLE IV
MODEL EVALUATION RESULTS

Model	MSE	MAE	RMSE	R ²
LightGBM	443,503,237,551.86	344,049.91	665,960.39	0.54
CatBoost	445,416,111,571.58	346,383.98	667,395.02	0.53
Linear Regression	487,196,432,706.98	380,498.59	697,994.58	0.49
Random Forest Regression	459,187,958,437.92	353,549.47	677,634.09	0.52
Neural Network	464,307,168,013.16	360,587.51	681,400.89	0.51

TABLE V
USE OF COMPUTING RESOURCES

Model	Execution Time (s)	Memory Usage (MB)	CPU Time (User)	CPU Time (System)
LightGBM	1.97	2.67	1.04	0.83
CatBoost	4.25	7.49	20.67	1.90
Linear Regression	0.27	100.12	0.58	0.06
Random Forest Regression	1.36	77.98	5.42	0.20
Neural Network	123.69	106.52	133.02	7.61

(Widodo et al., 2025)

Table 5. Comparison between the performance of XGBoost and CatBoost with several machine learning models.

Algorithm	R ²	MSE	RMSE	MAE
CatBoost	0.990	57874.073	229.152	110.345
XGBoost	0.984	66962.798	243.538	121.858
CART	0.978	225875.636	444.333	154.754
AdaBoost	0.984	146579.19	372.080	249.915
GB	0.994	79287.530	267.877	109.291
RF	0.982	293742.469	9501.507	209.865
LightGBM	0.990	89441.261	287.131	125.247
NN	0.962	547184.31	287.431	298.015
SVM	0.976	247092.407	488.957	286.563

(Mahesh & Soundrapandiyan, 2024)

Implementación del modelo

En el archivo 05_price_mode.ipynb se configura y lleva a cabo el entrenamiento del modelo.

Configuración:

- 500 épocas
- 5-fold CV, se divide el dataset en 5, 4 son para entrenamiento y 1 para prueba, después se intercambia el set de prueba. Al final se toma el promedio de cada fold
- Learning_rate = 0.05
- Máxima profundidad del árbol 5, recomendada para datasets pequeños y reducir overfitting
- l2_leaf_reg=5, parámetro de regularización L2, entre 1 y 5 si el modelo tiene desempeño deficiente y entre 5 y 10 para reducir overfitting.
- Función de pérdida MAE (Mean Absolut Error)

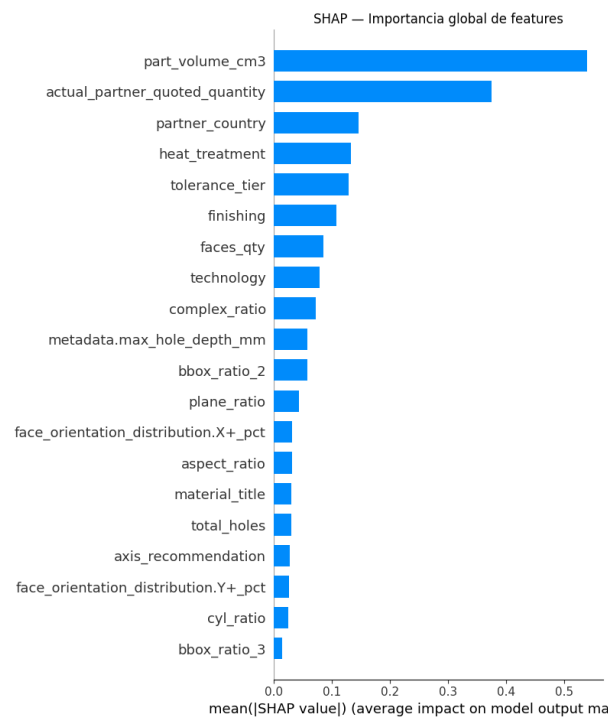
- Métrica MAPE (Error Porcentual Absoluto Medio)

Dado que una misma pieza puede tener múltiples partners que la cotizaron y estas cotizaciones pueden variar, se hicieron dos entrenamientos: Uno que únicamente considera la cotización del partner que se tomó para el proyecto y otro que toma todas las cotizaciones. El modelo obtuvo mejores resultados cuando se consideraron todas las cotizaciones (10 puntos MAPE de mejora) pues lógicamente tenía un número más elevado de registros.

Resultados

Entrenamiento: 32.96%

MAPE: 60.23% $\pm$ 10.21% (5-fold)



Conclusiones

- El modelo tiene overfitting pues en el entrenamiento presenta mejores resultados que en la validación
- La mejora del modelo con el aumento de los registros podría indicar que la principal causa de que el modelo no sea más preciso es la falta de datos.
- No se continuo con la mejora de este enfoque pues me di cuenta que desde el inicio el objetivo del modelo estaba mal diseñado. La existencia de diferentes valores en la variable objetivo para el mismo conjunto de variables independientes provoca que el error esté determinado por la variación de cotizaciones de los partners. Aunque la

inferencia del modelo para una pieza con 4 cotizaciones acierte exactamente en una de ellas, el error no disminuiría pues estaría más alejado de las otras 3.

- La imagen de feature importance indica que el modelo no asigna mucha relevancia a características individuales de la pieza, sino más a factores generales como cantidad de piezas en la orden, país donde se manufactura, el tamaño, acabados y post-procesamientos.

Segunda iteración

Objetivo: Inferir el precio unitario de la pieza que Radii ofrece al cliente.

Justificación: A diferencia de querer inferir el precio cotizado por el partner en el que pueden existir diferentes cotizaciones que pueden variar hasta en más de un 100%, el precio ofrecido por Radii al cliente es solo 1 y depende de las mismas variables independientes.

Carpeta: iter2

Cambios respecto a la iteración anterior:

- Cambio en el objetivo del modelo
- Cambio en la métrica de desempeño del modelo (de MAPE a Median Error Percentage en los percentiles 50, 90 y 99)
- Cambio de variable objetivo y su normalización
- Mejoras en la configuración del modelo para disminuir overfitting

Extracción de los datos

La extracción de los datos hizo uso de las herramientas y procesos utilizados en la iteración uno. Únicamente se ajustó la herramienta de exportación de datos de la plataforma para que exporte las mismas columnas que incluía la sección de *manufacturing quotes* (cotizaciones de los partners), para añadirlas en *lineItems* que son las piezas que contiene una orden (Una orden puede contener múltiples piezas).

Transformación de los datos

La transformación de los datos se hizo en 01_cleanning.ipynb de una forma más estructurada aunque sigue siendo muy similar a como se hizo en la iteración 1: Quitar registros sin información, quitar columnas que no sirvan al objetivo del modelo, quitar columnas que no tengan variación en los datos, quitar registros que sean outliers, quitar columnas con correlaciones muy altas y poner las variables categóricas en el formato que el modelo necesita. Al final solo normalizamos la variable objetivo esta vez con $\ln(x)$ y esta vez si separamos los datos 80% para entrenamiento y 20% para prueba.

Debido a que muchas veces no se registran en la plataforma las cotizaciones de los partners pero desde que existe la plataforma sí se registra el precio que se le da al cliente el dataset era bastante más grande. De alrededor de 4000 registros. Al término de la transformación nos quedaron 386 registros. La principal pérdida de información es nuevamente debido a que la mayoría de ordenes únicamente tiene planos en 2D y de todas las que tienen 3D, no se ha corrido en muchas el DFM análisis por ser un procedimiento costoso.

Columnas utilizadas, 6 categóricas, 14 numéricas: Material, Technology, Tolerance, Post Production, Finishing, Heat_Treatment, Quantity, Unit_Price_(USD), part_volume_cm3, stock_volume_cm3, material_removal_pct, stock_dimensions_mm.x, stock_dimensions_mm.y, stock_dimensions_mm.z, faces_qty, aspect_ratio, thin_ratio, plane_ratio, cyl_ratio, complex_ratio, total_holes, part_type, log_unit_price, log_quantity

Carga de los datos

Para cargar los datos al modelo eficientemente se pueden reutilizar los notebooks y extraer los datos de la herramienta de extracción que se hizo en la plataforma de la organización.

Modelado

Sobre el modelo

Nuevamente se utilizó catboost. La información de este modelo se encuentra redactada en la sección de Modelado de la iteración 1.

Implementación del modelo

La implementación se encuentra en el archivo 02_modeling.ipynb.

Configuración:

- Regularización L2 (Ridge) sobre hojas con coeficiente 10 para reducir overfitting agresivamente.
- Subsampling Bernoulli al 80% por árbol, excluye el 20% de los registros evitando tomar siempre el más determinante para la creación de los árboles. Reduce overfitting.
- Random_strength = 3, reduce overfitting aumentando la aleatoriedad en la selección de variables determinantes para generar los árboles.
- Entrenamiento de hasta 3000 árboles con early stopping (100 rondas sin mejora),
- Learning_rate = 0.03
- Profundidad de los árboles = 4.
- Validación con 5-fold
- Test con 20% de datos reservados.
- Función de pérdida MAE (Mean Absolut Error)

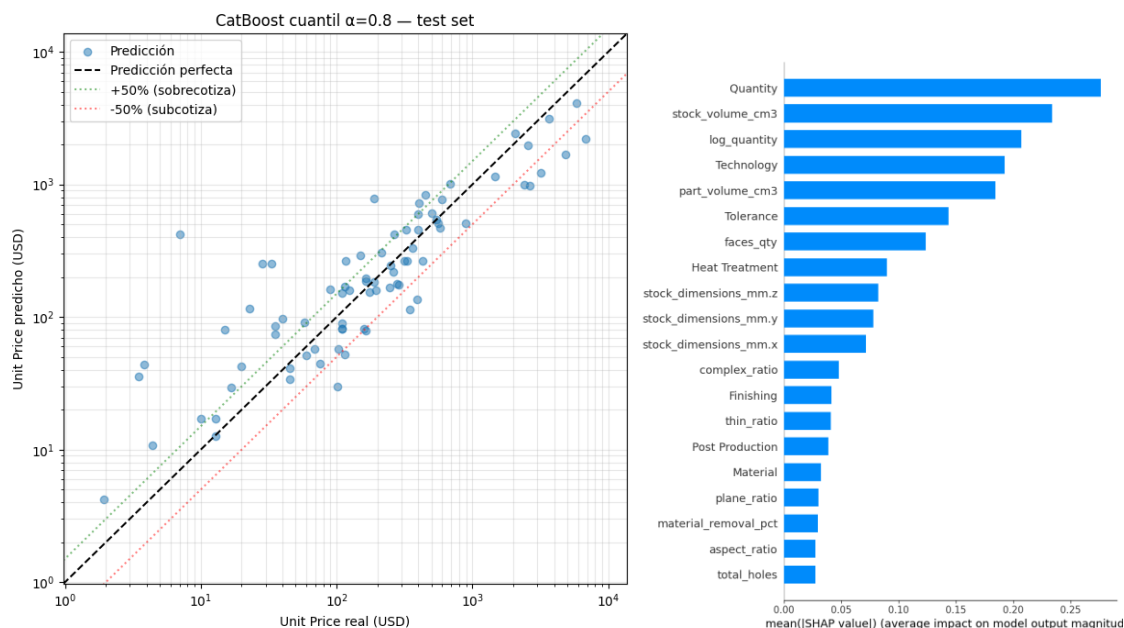
- Métricas reportadas: P50/P90/P99 (se ordenan los porcentajes de errores de la inferencia respecto a la real de menor a mayor y observamos las posiciones al 50%, 90% y 99% de la fila. Esta métrica es mejor para entender el comportamiento de las inferencias. Anteriormente con el MAPE un error muy alto podía inflarlo demasiado, aunque la mayoría de las predicciones fueran aceptables. Con esta métrica podemos ver el error de la mayoría de predicciones y el que corresponde a los casos raros y extremos, (Spiliotis et al., 2019)).

Resultados

Entrenamiento: P50/P90/P99 - 17.7% / 96.4% / 790.2%

Validación (Promedio de 5-folds): P50/P90/P99 - 41.12%/215.1%/2336%

Prueba: P50/P90/P99 - 43.3% / 197.4% / 2168.5%



Con el objetivo de hacer un cotizador que dependa únicamente de variables humanamente extraíbles de planos, se realizó un tercer intento eliminando aquellas columnas que este análisis detecta que no son tan relevantes, los resultados fueron prácticamente los mismos. Su proceso de ETL e implementación están en los notebooks 03_dataset_2d.ipynb y 04_modeling_2d.ipynb respectivamente.

Conclusiones

- A pesar de la configuración de parámetros agresiva para solucionar el overfitting, este no ha podido ser resuelto
- El desempeño del modelo no está listo para ser un modelo utilizable en un entorno de producción (P50/P90/P99 - 15% / 30% / 100%)

- El que las variables de mayor impacto en la inferencia del precio sean poco descriptivas de las piezas en específico, en un inicio me entusiasmó porque pensaba que existía la posibilidad de poder hacer un modelo con variables fáciles de obtener, sin embargo, más tarde entendí que en realidad es un problema de los datos que no cuentan con suficientes casos para identificar cómo una geometría complicada puede repercutir en gran medida a los precios tal cómo sucede en la realidad.
- No hay variabilidad en columnas importantes como Material y Tecnología. La mitad de los registros tienen “other” cómo valor (47% de los registros) o “CNC 5 axis” en el caso de tecnología (94% de los registros). Esto hace que el modelo no pueda identificar la relevancia de los materiales y la tecnología utilizada para calcular el precio.

Próximos pasos

- Se han planeado realizar tareas para aumentar el número de registros
 - Extraer datos de los proyectos realizados por el partner con el que tenemos mejor relación.
 - Usar los cotizadores de las empresas competencia para obtener datos de precios y validar nuestra competitividad
 - Optimización de procesos internos para una captura más constante y completa de la información de los proyectos
- Se están trabajando cambios en la plataforma para añadir otras variables que en la realidad impactan en gran medida el precio de las piezas como tolerancias geométricas, tipos de roscados, cambios de herramientas etc.
- Se harán cambios en la plataforma para que en lugar de poner “other” en ciertos parámetros, haya más opciones válidas y se ponga el texto adjunto a “other” para posteriormente normalizarlo con un agente de IA.

Consideraciones

- Los datos utilizados para los modelos de ML al ser propiedad de Radii no serán subidos a repositorios ni entregados como evidencia de la actividad. Sin embargo, en los notebooks se encuentra resumida en métricas la información que contiene el dataset.
- Todo el código utilizado fue generado por el agente de inteligencia artificial de Claude Anthropic.
- Se obtuvo el permiso de la organización para poder presentar esta información.

Bibliografía

Boncinelli, L., et al. (2025). Mambular: A sequential model for tabular deep learning. OpenReview. <https://openreview.net/forum?id=wElgE9qBb5>

Widodo, S., Utomo, F. S., & Berlilana. (2025). A comprehensive evaluation of CatBoost and LightGBM algorithms for honorarium prediction on categorical datasets with class imbalance. JUITA: Jurnal Informatika, 13(3), 359–370. <https://doi.org/10.30595/juita.v13i3.27363>

Mahesh, P., & Soundrapandiyan, R. (2024). Yield prediction for crops by gradient-based algorithms. PLOS ONE, 19(8), e0291928. <https://doi.org/10.1371/journal.pone.0291928>

Spiliotis, E., Nikolopoulos, K., & Assimakopoulos, V. (2019). Tales from tails: On the empirical distributions of forecasting errors and their implication to risk. International Journal of Forecasting, 35(2), 687–698. <https://doi.org/10.1016/j.ijforecast.2018.10.004>

Gneiting, T., Wolfram, D., Resin, J., Kraus, K., Bracher, J., Dimitriadis, T., Hagenmeyer, V., Jordan, A. I., Lerch, S., Phipps, K., & Schienle, M. (2023). Model diagnostics and forecast evaluation for quantiles. Annual Review of Statistics and Its Application, 10, 597–621. <https://doi.org/10.1146/annurev-statistics-032921-020240>